

SQLite

SQLite (/ˌɛskjuːˌɛlˈaɪt/ "S-Q-L-ite",^{[4][5]} /ˈsiːkwəˌlaɪt/ "sequel-ite"^[6]) is a free and open-source relational database engine written in the C programming language. It is not a standalone application; rather, it is a library that software developers embed in their applications. As such, it belongs to the family of embedded databases. According to its developers, SQLite is the most widely deployed database engine, as it is used by several of the top web browsers, operating systems, mobile phones, and other embedded systems.^[7]

Many programming languages have bindings to the SQLite library. It generally follows PostgreSQL syntax, but does not enforce type checking by default.^{[8][9]} This means that one can, for example, insert a string into a column defined as an integer. Although it is a lightweight embedded database, SQLite implements most of the SQL standard and the relational model, including transactions and ACID guarantees.^[10] However, it omits many features implemented by other databases, such as materialized views and complete support for triggers and ALTER TABLE statements.^[11]

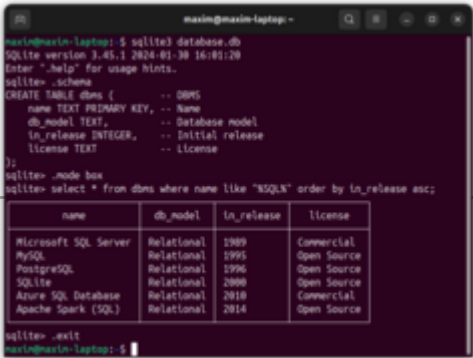
History

D. Richard Hipp designed SQLite in the spring of 2000 while working for General Dynamics on contract with the United States Navy.^[12] Hipp was designing software used for a damage-control system aboard guided-missile destroyers; the damage-control system originally used HP-UX with an Informix database back-end. SQLite began as a Tcl extension.^[13]

In August 2000, version 1.0 of SQLite was released, with storage based on gdbm (GNU Database Manager). In September 2001, SQLite 2.0 replaced gdbm with a custom B-tree implementation,^[a] adding transaction capability. In June 2004, SQLite 3.0 added internationalization, manifest typing, and

SQLite





Screenshot of sqlite3 command-line shell program

Developer	D. Richard Hipp
Release	17 August 2000
Stable release	3.53.4 ^[1]  (24 July 2026)
Written in	C
Operating system	Cross-platform
Size	699 KiB
Type	RDBMS (embedded)
License	Public domain ^[2]
Website	sqlite.org (https://sqlite.org) 
Repository	sqlite.org/src (https://sqlite.org/src)

SQLite Database File Format	
Filename extension	.sqlite, .sqlite3, .db, .db3, .s3db, .sl3
Internet media type	application/vnd.sqlite3 ^[3]
Magic number	53 51 4c 69 74 65 20 66 6f 72 6d 61 74 20 33 00

other major improvements, partially funded by America Online. In 2011, Hipp announced his plans to add a NoSQL interface to SQLite, as well as announcing UnQL, a functional superset of SQL designed for document-oriented databases.^[15]

SQLite is one of four formats recommended for long-term storage of datasets approved for use by the United States Library of Congress.^{[16][17][18]}

	(zero-terminated <u>ASCII</u> "SQLite format 3")
Initial release	2004-06-18
Open format?	yes (<u>Public Domain</u>)
Website	<u>sqlite.org/fileformat.html</u> (<u>https://sqlite.org/fileformat.html</u>)

Design

SQLite was designed to allow the program to be operated without installing a database management system or requiring a database administrator. Unlike client-server database management systems, the SQLite engine has no standalone processes with which the application program communicates. Instead, a linker integrates the SQLite library—statically or dynamically—into an application program which uses SQLite's functionality through simple function calls, reducing latency in database operations; for simple queries with little concurrency, SQLite performance profits from avoiding the overhead of inter-process communication.

Due to the serverless design, SQLite applications require less configuration than client-server databases. SQLite is called *zero-configuration*^[19] because configuration tasks such as service management, startup scripts, and password- or GRANT-based access control are unnecessary. Access control is handled through the file-system permissions of the database file.^[20] Databases in client-server systems use file-system permissions that give access to the database files only to the daemon process, which handles its locks internally, allowing concurrent writes from several processes.

SQLite stores the entire database, consisting of definitions, tables, indices, and data, as a single cross-platform file, allowing several processes or threads to access the same database concurrently. It implements this simple design by locking the database file during writing.^[20] Write access may fail with an error code, or it can be retried until a configurable timeout expires. SQLite read operations can be multitasked, though due to the serverless design, writes can only be performed sequentially. This concurrent access restriction does not apply to temporary tables, and it is relaxed in version 3.7 as write-ahead logging (WAL) enables concurrent reads and writes.^[21] Since SQLite has to rely on file-system locks, it is not the preferred choice for write-intensive deployments.^[22]

SQLite uses PostgreSQL as a reference platform. "What would PostgreSQL do" is used to make sense of the SQL standard.^{[23][24]} One major deviation is that, with the exception of primary keys, SQLite does not enforce type checking; the type of a value is dynamic and not strictly constrained by the schema (although the schema will trigger a conversion when storing, if such a conversion is potentially reversible). SQLite strives to follow Postel's rule.^[25]

Features

SQLite implements most of the SQL-92 standard for SQL, but lacks some features. For example, it only partially provides triggers and cannot write to views (however, it provides INSTEAD OF triggers that provide this functionality). Its support of ALTER TABLE statements is limited.^[26]

SQLite uses an unusual type system for an SQL-compatible DBMS: instead of assigning a type to a column as in most SQL database systems, types are assigned to individual values; in language terms it is *dynamically typed*. Moreover, it is *weakly typed* in some of the same ways that Perl is: one can insert a string into an integer column (although SQLite will try to convert the string to an integer first, if the column's preferred type is integer). This adds flexibility to columns, especially when bound to a dynamically typed scripting language. However, the technique is not portable to other SQL products. A common criticism is that SQLite's type system lacks the data integrity mechanism provided by statically typed columns, although it can be emulated with constraints like **CHECK**(typeof(x)='integer').^[12] In 2021, support for static typing was added through STRICT tables, which enforce datatype constraints for columns.^[27]

Tables normally include a hidden *rowid* index column, which provides faster access.^[28] If a table includes an INTEGER PRIMARY KEY column, SQLite will typically optimize it by treating it as an alias for the *rowid*, causing the contents to be stored as a strictly typed 64-bit signed integer and changing its behavior to be somewhat like an auto-incrementing column. SQLite includes an option to create a table without a rowid column, which can save disk space and improve lookup speed. WITHOUT ROWID tables are required to have a primary key.^[29]

SQLite supports foreign key constraints,^{[30][31]} although they are disabled by default and must be manually enabled with a PRAGMA statement.^[32]

Stored procedures are not supported; this is an explicit choice by the developers to favor simplicity, as the typical use case of SQLite is to be embedded inside a host application that can define its own procedures around the database.^[33]

SQLite does not have full Unicode support by default for backwards compatibility and due to the size of the Unicode tables, which are larger than the SQLite library.^[34] Full support for Unicode case-conversions can be enabled through an optional extension.^[35]

SQLite supports full-text search through its FTS5 loadable extension, which allows users to efficiently search for a keyword in a large number of documents similar to how search engines search webpages.^[36]

SQLite includes support for working with JSON through its *json1* extension, which is enabled by default since 2021. SQLite's JSON functions can handle JSON5 syntax since 2023. In 2024, SQLite added support for JSONB, a binary serialization of SQLite's internal representation of JSON. Using JSONB allows applications to avoid having to parse the JSON text each time it is processed and saves a small amount of disk space.^[37]

In May 2025, the 25th-anniversary release SQLite 3.50.0 introduced additional features, including new Unicode functions (`unistr()` and `unistr_quote()`), a new API (`sqlite3_setlk_timeout()`) for setting lock timeouts, improved command-line tools and rsync utility enhancements, and optimized JSONB.^[38]

The maximum supported size for an SQLite database file is 281 terabytes.^[39]

SQLite normally stores every table and index in a SQLite database in a single ordinary disk file. Alternatively, SQLite can be told to keep a SQL database entirely in memory as an in-memory database with the `:memory:` connection string.^[40]

Development and distribution

SQLite's code is hosted with Fossil, a distributed version control system that uses SQLite as a local cache for its non-relational database format, and SQLite's SQL as an implementation language.^{[41][42]}

SQLite is public domain, but not "open-contribution", with the website stating "the project does not accept patches from people who have not submitted an affidavit dedicating their contribution into the public domain."^[43] Instead of a code of conduct, the founders have adopted a code of ethics based on chapter 4 of the Rule of St. Benedict.^[44]

A standalone command-line shell program called *sqlite3*^[45] is provided in SQLite's distribution. It can be used to create a database, define tables, insert and change rows, run queries and manage an SQLite database file. It also serves as an example for writing applications that use the SQLite library.

SQLite uses automated regression testing prior to each release. Over 2 million tests are run as part of a release's verification. The SQLite library has 156,000 lines of source code, while all the test suites combined add up to 92 million lines of test code. SQLite's tests simulate a number of exceptional scenarios, such as power loss and I/O errors, in addition to testing the library's functionality. Starting with the August 10, 2009 release of SQLite 3.6.17, SQLite releases have 100% branch test coverage, one of the components of code coverage. SQLite has four different test harnesses: the original public-domain TCL tests, the proprietary C-language TH3 test suite, the SQL Logic Tests, which check SQLite against other SQL databases, and the dbsqlfuzz proprietary fuzzing engine.^[46]

Notable uses

Operating systems

SQLite is included by default in:

- Android^{[14][47]}
- BlackBerry Tablet OS^[48]
- iOS^[13]
- Mac OS X 10.4 onwards (Apple adopted it as an option in macOS's Core Data API from the original implementation)^[13]
- NetBSD^[49]
- NixOS where it is used by the Nix core package management system^[50]
- OpenBSD, which supports exporting ports tree metadata as an SQLite database^[51]
- RPM based Linux distributions (including Red Hat Enterprise Linux and its derivative Fedora Linux), where it is used by the core package management system^[52]
- Windows 10 onwards^[53]

Middleware

- ADO.NET adapter, initially developed by Robert Simpson, is maintained jointly with the SQLite developers since April 2010.^[54]
- ODBC driver has been developed and is maintained separately by Christian Werner.^[55] Werner's ODBC driver is the recommended connection method for accessing SQLite from OpenOffice.org.^[56]
- COM (ActiveX) wrapper making SQLite accessible on Windows to scripted languages such as JScript and VBScript. This adds SQLite database capabilities to HTML Applications (HTA).^[57]

Web browsers

- The browsers Google Chrome, Opera, Safari and the Android Browser all allow for storing information in, and retrieving it from, an SQLite database within the browser, using the official SQLite Wasm (WebAssembly) build,^[58] or using the Web SQL Database technology, although the latter is becoming deprecated (namely superseded by SQLite Wasm or by IndexedDB). Internally, Chromium based browsers use SQLite databases for storing configuration data like site visit history, cookies, download history etc.^[59]
- Mozilla Firefox and Mozilla Thunderbird store a variety of configuration data (bookmarks, cookies, contacts etc.) in internally managed SQLite databases. Until Firefox version 57 ("Firefox Quantum"), there was a third-party add-on that used the API supporting this functionality to provide a user interface for managing arbitrary SQLite databases.^[60]
- Several third-party add-ons can make use of JavaScript APIs to manage SQLite databases.^{[61][62]}

Web application frameworks

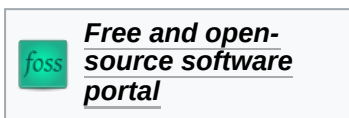
- Symfony
- Laravel
- Bugzilla
- Django's default database management system
- Drupal
- Trac
- Ruby on Rails's default database management system
- web2py

Others

- Adobe Systems uses SQLite as its file format in Adobe Lightroom, a standard database in Adobe AIR, and internally within Adobe Acrobat Reader.^[13]
- Apple Photos uses SQLite internally.^[63]
- Audacity uses SQLite as its file format, as of version 3.0.0.^[64]
- Evernote uses SQLite to store its local database repository in Windows.
- Skype^[65]
- WhatsApp^[66]

- The Service Management Facility, used for service management within the Solaris and OpenSolaris operating systems
- Dropbox client software^[13]
- Intuit uses SQLite in QuickBooks and TurboTax^[13]
- McAfee Antivirus^[13]
- Flame (malware)
- BMW iDrive satellite navigation system
- TomTom GPS systems, for the NDS map data
- Proxmox VE - *Proxmox Cluster File System* (pmxcfs ([https://pve.proxmox.com/wiki/Proxmox_Cluster_File_System_\(pmxcfs\)\)\)](https://pve.proxmox.com/wiki/Proxmox_Cluster_File_System_(pmxcfs)))))
- Bentley Systems MicroStation^[13]
- Bosch car multimedia systems^[13]
- Airbus A350 flight system^[13]
- Quicken Essentials and later versions of Quicken for Mac^[67]
- Python standard library^[68]
- Xojo IDE^[13]
- Wine^[69]

See also



- Ordered key–value store
- Comparison of relational database management systems
- List of public domain projects
- List of relational database management systems
- MySQL
- Spatialite

Notes

- a. SQLite's B-tree implementation was originally adapted from The Art of Computer Programming.^[14]

References

Citations

1. "SQLite Release 3.53.4 On 2026-07-24" (https://sqlite.org/releaselog/3_53_4.html). Retrieved 25 July 2026.
2. "SQLite Copyright" (<https://sqlite.org/copyright.html>). sqlite.org. Retrieved May 17, 2010.

3. "SQLite database file format media type at IANA" (<https://www.iana.org/assignments/media-types/application/vnd.sqlite3>). *Internet Assigned Numbers Authority*. IANA. Retrieved 2019-03-08.
4. "Why SQLite succeeded as a database — Richard Hipp, creator of SQLite" (<https://changelog.com/podcast/201>). *The Changelog*. Episode 201. Event occurs at 00:16:00. Archived (<https://web.archive.org/web/20220707033506/https://changelog.com/podcast/201>) from the original on 2022-07-07. Retrieved 2025-04-11. "How do I pronounce the name of the product? I say S-Q-L-ite, like a mineral."
5. D. Richard Hipp (presenter) (May 31, 2006). *An Introduction to SQLite* (<https://www.youtube.com/watch?v=f428dSRkTs4#t=1m14s>) (video). Google Inc. Event occurs at 00:01:14. Retrieved March 23, 2010. "[ɛs,kju,wəl'art]"
6. D. Richard Hipp (presenter) (May 31, 2006). *An Introduction to SQLite* (<https://www.youtube.com/watch?v=f428dSRkTs4#t=48m15s>). Google Inc. Event occurs at 00:48:15. Retrieved March 23, 2010. "[ˈsɪkwə,lart]"
7. "Most Widely Deployed SQL Database Estimates" (<https://sqlite.org/mostdeployed.html>). SQLite.org. Retrieved May 11, 2011.
8. Owens, Michael (2006). "Chapter 4: SQL" (<https://books.google.com/books?id=VsZ5bUh0XAkC&pg=PA133>). In Gilmore, Jason; Thomas, Keir (eds.). *The Definitive Guide to SQLite*. D. Richard Hipp (foreword), Preston Hagar (technical reviewer). Apress. p. 133. ISBN 978-1-59059-673-9. Archived (<https://web.archive.org/web/20201124002058/https://books.google.com/books?id=VsZ5bUh0XAkC&pg=PA133>) from the original on 24 November 2020. Retrieved 30 December 2014.
9. "STRICT Tables" (<https://sqlite.org/stricttables.html>). Archived (<https://web.archive.org/web/20220807204905/https://sqlite.org/stricttables.html>) from the original on 2022-08-07. Retrieved 2022-08-11.
10. "Full-Featured SQL" (<https://www.sqlite.org/fullsql.html>). SQLite. Retrieved January 24, 2025.
11. "SQL Features That SQLite Does Not Implement" (<https://www.sqlite.org/omitted.html>). SQLite. Retrieved January 24, 2025.
12. Owens, Michael (2006). "Introducing SQLite". *The Definitive Guide to SQLite*. Apress. pp. 1–16. doi:10.1007/978-1-4302-0172-4_1 (https://doi.org/10.1007%2F978-1-4302-0172-4_1). ISBN 978-1-59059-673-9.
13. "Well-Known Users Of SQLite" (<https://sqlite.org/famous.html>). SQLite. Archived (<https://web.archive.org/web/20150711135311/https://sqlite.org/famous.html>) from the original on July 11, 2015. Retrieved August 5, 2015.
14. Bell, Adam. "The Untold Story of SQLite" (<https://corecursive.com/066-sqlite-with-richard-hipp/>). *Corecursive*. Retrieved 16 November 2025.
15. "Interview: Richard Hipp on UnQL, a New Query Language for Document Databases" (<http://www.infoq.com/news/2011/08/UnQL>). InfoQ. August 4, 2011. Archived (<https://web.archive.org/web/20140408215240/http://www.infoq.com/news/2011/08/UnQL>) from the original on April 8, 2014. Retrieved October 5, 2011.
16. "LoC Recommended Storage Format" (<https://sqlite.org/locrsf.html>). *sqlite.org*. Archived (<https://web.archive.org/web/20200423212849/https://sqlite.org/locrsf.html>) from the original on 2020-04-23. Retrieved 2020-04-09.
17. "SQLite, Version 3" (<https://www.loc.gov/preservation/digital/formats/fdd/fdd000461.shtml>). *www.loc.gov*. 2017-03-28. Archived (<https://web.archive.org/web/20200511194518/https://www.loc.gov/preservation/digital/formats/fdd/fdd000461.shtml>) from the original on 2020-05-11. Retrieved 2020-04-09.

18. "Recommended Formats Statement – datasets/databases" (<https://www.loc.gov/preservation/resources/rfs/data.html>). Library of Congress. Archived (<https://web.archive.org/web/20180822113435/https://www.loc.gov/preservation/resources/rfs/data.html>) from the original on 2018-08-22. Retrieved 2020-04-09.
19. "SQLite Is A Zero-Configuration Database" (<https://sqlite.org/zeroconf.html>). SQLite.org. Archived (<https://web.archive.org/web/20240502210736/https://sqlite.org/zeroconf.html>) from the original on May 2, 2024. Retrieved August 3, 2015.
20. "SQLite" (<https://clickhouse.com/docs/en/engines/database-engines/sqlite>). *ClickHouse Docs*. Retrieved January 25, 2025.
21. "Write Ahead Logging in SQLite 3.7" (<https://sqlite.org/wal.html>). SQLite.org. Archived (<https://web.archive.org/web/20240502210711/https://sqlite.org/wal.html>) from the original on May 2, 2024. Retrieved September 3, 2011. "WAL provides more concurrency as readers do not block writers and a writer does not block readers. Reading and writing can proceed concurrently."
22. "Appropriate Uses For SQLite" (<https://sqlite.org/whentouse.html>). SQLite.org. Archived (<https://web.archive.org/web/20240502210713/https://sqlite.org/whentouse.html>) from the original on 2024-05-02. Retrieved 2015-09-03.
23. Berkus, Josh (4 June 2014). "PGCon 2014: Clustering and VODKA" (<https://lwn.net/Articles/601144/>). *Lwn.net*. Archived (<https://web.archive.org/web/20150629195442/https://lwn.net/Articles/601144/>) from the original on 2015-06-29. Retrieved 2017-01-06.
24. "PGCon2014: SQLite: Protégé of PostgreSQL" (<https://www.pgcon.org/2014/schedule/event/s/736.en.html>). *Pgcon.org*. 20 September 2015. Archived (<https://web.archive.org/web/20141230193958/http://www.pgcon.org/2014/schedule/events/736.en.html>) from the original on 2014-12-30. Retrieved 2017-01-06.
25. "SQLite: StrictMode" (<https://sqlite.org/src/wiki?name=StrictMode>). *Sqlite.org*. Archived (<https://web.archive.org/web/20160304115940/https://sqlite.org/src/wiki?name=StrictMode>) from the original on March 4, 2016. Retrieved September 3, 2015.
26. "Release History of SQLite" (<https://sqlite.org/changes.html>). Archived (<https://web.archive.org/web/20210316043517/https://sqlite.org/changes.html>) from the original on 2021-03-16. Retrieved 2021-03-22.
27. "STRICT Tables" (<https://sqlite.org/stricttables.html>). *SQLite*. Retrieved January 24, 2025.
28. "SQL As Understood By SQLite" (https://sqlite.org/lang_createtable.html#rowid). *SQLite*. Archived (https://web.archive.org/web/20180521104530/https://sqlite.org/lang_createtable.html#rowid) from the original on 21 May 2018. Retrieved 21 May 2018. "Searching for a record with a specific rowid, or for all records with rowids within a specified range is around twice as fast as a similar search made by specifying any other PRIMARY KEY or indexed value."
29. "Clustered Indexes and the WITHOUT ROWID Optimization" (<https://sqlite.org/withoutrowid.html>). *SQLite*. Retrieved January 24, 2025.
30. Karwin, Bill (May 2010). Carter, Jacquelyn (ed.). *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. The Pragmatic Bookshelf. p. 70. ISBN 978-1-934356-55-5. "Sometimes you're forced to use a database brand that doesn't support foreign key constraints (for example MySQL's MyISAM storage engine or SQLite prior to version 3.6.19)."
31. "SQLite Release 3.6.19 On 2009-10-14" (https://sqlite.org/releaselog/3_6_19.html). *sqlite.org*. Archived (https://web.archive.org/web/20201029060401/http://sqlite.org/releaselog/3_6_19.html) from the original on 2020-10-29. Retrieved 2020-10-15.
32. "SQLite Foreign Key Support" (<https://sqlite.org/foreignkeys.html>). *SQLite*. Retrieved January 24, 2025.
33. Source: developers' comments on [SQLite forum](https://sqlite.org/forum/info/78a60bdeec7c1ee9) (<https://sqlite.org/forum/info/78a60bdeec7c1ee9>) Archived (<https://web.archive.org/web/20230401220416/https://sqlite.org/forum/info/78a60bdeec7c1ee9>) 2023-04-01 at the [Wayback Machine](https://www.waybackmachine.org/)

34. "Quirks, Caveats, and Gotchas In SQLite" (<https://sqlite.org/quirks.html>). *SQLite*. Retrieved January 24, 2025.
35. "Case-insensitive matching of Unicode characters does not work" (<https://sqlite.org/faq.html#q18>). *SQLite Frequently Asked Questions*. Archived (<https://web.archive.org/web/20150905054749/https://sqlite.org/faq.html#q18>) from the original on 2015-09-05. Retrieved 2015-09-03.
36. "SQLite FTS5 Extension" (<https://sqlite.org/fts5.html>). *SQLite*. Retrieved January 24, 2025.
37. "JSON Functions And Operators" (<https://sqlite.org/json1.html>). *SQLite*. Retrieved January 24, 2025.
38. "SQLite Release 3.50.0 On 2025-05-29" (https://sqlite.org/releaselog/3_50_0.html). *SQLite*. Retrieved November 3, 2025.
39. "Limits In SQLite" (<https://sqlite.org/limits.html>). *SQLite.org*. Archived (<https://web.archive.org/web/20211107064937/https://sqlite.org/limits.html>) from the original on 2021-11-07. Retrieved 2022-09-19.
40. "In-Memory Databases" (<http://www.sqlite.org/inmemorydb.html>). *www.sqlite.org*.
41. "Thoughts On The Design Of The Fossil DVCS" (<https://www.fossil-scm.org/home/doc/trunk/www/theory1.wiki>). Fossil-scm.org. July 12, 2017. Archived (<https://web.archive.org/web/20221013234319/https://www.fossil-scm.org/home/doc/trunk/www/theory1.wiki>) from the original on October 13, 2022. Retrieved October 14, 2022.
42. "Fossil: Fossil Performance" (<https://www.fossil-scm.org/index.html/doc/tip/www/stats.wiki>). Fossil-scm.org. August 23, 2009. Archived (<https://web.archive.org/web/20091009054952/http://www.fossil-scm.org/index.html/doc/tip/www/stats.wiki>) from the original on October 9, 2009. Retrieved September 12, 2009.
43. "SQLite Copyright" (<https://sqlite.org/copyright.html>). *sqlite.org*. Archived (<https://web.archive.org/web/20240315172355/https://sqlite.org/copyright.html>) from the original on 2024-03-15. Retrieved 2024-03-06.
44. "Code Of Ethics" (<https://sqlite.org/codeofethics.html>). *sqlite.org*. Archived (<https://web.archive.org/web/20240219225117/https://sqlite.org/codeofethics.html>) from the original on 2024-02-19. Retrieved 2024-03-06.
45. "Command Line Shell For SQLite" (<https://sqlite.org/cli.html>). *Sqlite.org*. Archived (<https://web.archive.org/web/20221006104551/https://sqlite.org/cli.html>) from the original on October 6, 2022. Retrieved October 14, 2022.
46. "How SQLite Is Tested" (<https://sqlite.org/testing.html>). *SQLite.org*. Archived (<https://web.archive.org/web/20091006224147/https://sqlite.org/testing.html>) from the original on October 6, 2009. Retrieved September 12, 2009.
47. "android.database.sqlite" (<https://developer.android.com/reference/android/database/sqlite/package-summary>). *Android Developers*. Retrieved 23 April 2026.
48. "Open Source Components for the Native SDK for BlackBerry Tablet OS" (<https://web.archive.org/web/20130127102233/http://blackberry.github.com/ndk/components.html>). Archived from the original (<https://blackberry.github.com/ndk/components.html>) on 2013-01-27. Retrieved 2017-09-19.
49. "sqlite3(1) - NetBSD Manual Pages" (<https://man.netbsd.org/sqlite3.1>). *man.netbsd.org*. NetBSD. Retrieved 23 April 2026.
50. "Prerequisites" (<https://nix.dev/manual/nix/2.28/installation/prerequisites-source.html>). *nix.dev*. NixOS. Retrieved 23 April 2026.
51. "ports(7) - OpenBSD manual pages" (<https://man.openbsd.org/ports>). *man.openbsd.org*. OpenBSD. Retrieved 23 April 2026.
52. "RPM Database Recovery" (https://rpm.org/user_doc/db_recovery.html). *rpm.org*. Retrieved 23 April 2026.

53. "To use the version of SQLite that is installed with Windows" (<https://docs.microsoft.com/en-us/windows/uwp/data-access/sqlite-databases#to-use-the-version-of-sqlite-that-is-installed-with-windows>). 20 October 2022. Archived (<https://web.archive.org/web/20220331170828/https://docs.microsoft.com/en-us/windows/uwp/data-access/sqlite-databases#to-use-the-version-of-sqlite-that-is-installed-with-windows>) from the original on 31 March 2022. Retrieved 31 March 2022.
54. "Home" (<http://system.data.sqlite.org/index.html/doc/trunk/www/index.wiki>). *System.Data.SQLite*. 2016-12-30. Archived (<https://web.archive.org/web/20140713080835/http://system.data.sqlite.org/index.html/doc/trunk/www/index.wiki>) from the original on 2014-07-13. Retrieved 2017-01-06.
55. "SQLite ODBC Driver" (<http://www.ch-werner.de/sqliteodbc/>). *Ch-werner.de*. 2016-12-01. Archived (<https://web.archive.org/web/20140626165719/http://www.ch-werner.de/sqliteodbc/>) from the original on 2014-06-26. Retrieved 2017-01-06.
56. "Using SQLite Database with OpenOffice.org : Version 2.0" (http://documentation.openoffice.org/HOW_TO/data_source/SQLite.pdf) (PDF). *Documentation.openoffice.org*. Archived (https://web.archive.org/web/20110928073029/http://documentation.openoffice.org/HOW_TO/data_source/SQLite.pdf) (PDF) from the original on 2011-09-28. Retrieved 2017-01-06.
57. "sqlite — Sqlite Wrappers" (<https://sqlite.org/cvstrac/wiki?p=SqliteWrappers>). SQLite.org. February 7, 2009. Archived (<https://web.archive.org/web/20090205225756/http://sqlite.org/cvstrac/wiki?p=SqliteWrappers>) from the original on February 5, 2009. Retrieved February 7, 2009.
58. "sqlite3 WebAssembly & JavaScript Documentation Index" (<https://sqlite.org/wasm>). SQLite. Archived (<https://web.archive.org/web/20240502210710/https://sqlite.org/wasm/doc/trunk/index.md>) from the original on 2024-05-02. Retrieved 2023-05-08.
59. "Location of Google Chrome history" (<https://www.foxtonforensics.com/browser-history-examiner/chrome-history-location>). *www.foxtonforensics.com*. 2020-10-06. Archived (<https://web.archive.org/web/20230228184524/https://www.foxtonforensics.com/browser-history-examiner/chrome-history-location>) from the original on 2023-02-28. Retrieved 2020-10-06.
60. "SQLite Manager :: Add-ons for Firefox" (<https://web.archive.org/web/20170102010658/https://addons.mozilla.org/en-US/firefox/addon/sqlite-manager/>). *Addons.mozilla.org*. 2015-02-28. Archived from the original (<https://addons.mozilla.org/en-US/firefox/addon/sqlite-manager/>) on 2017-01-02. Retrieved 2017-01-06.
61. "SQLite Manager – Get this Extension for 🦊 Firefox (en-US)" (<https://addons.mozilla.org/en-US/firefox/addon/sqlite-manager-webext/>). *Addons.mozilla.org*. 2018-07-24. Archived (<https://web.archive.org/web/20181005112443/https://addons.mozilla.org/en-US/firefox/addon/sqlite-manager-webext/>) from the original on 2018-10-05. Retrieved 2018-10-05.
62. "SQLite Reader – Get this Extension for 🦊 Firefox (en-US)" (<https://addons.mozilla.org/en-US/firefox/addon/sql-reader/>). *Addons.mozilla.org*. 2018-09-01. Archived (<https://web.archive.org/web/20181005112536/https://addons.mozilla.org/en-US/firefox/addon/sql-reader/>) from the original on 2018-10-05. Retrieved 2018-10-05.
63. "Using SQL to find my best photo of a pelican according to Apple Photo" (<https://simonwillison.net/2020/May/21/dogsheep-photos/>). *Simon Willison's Weblog*. Archived (<https://web.archive.org/web/20200522181550/https://simonwillison.net/2020/May/21/dogsheep-photos/>) from the original on May 22, 2020. Retrieved May 23, 2020.
64. "Audacity 3.0.0 Released" (<https://web.archive.org/web/20230814021313/https://www.audacityteam.org/audacity-3-0-0-released/>). 17 March 2021. Archived from the original (<https://www.audacityteam.org/audacity-3-0-0-released/>) on 14 August 2023. Retrieved March 17, 2021.
65. Hinegardner, Jeremy (August 28, 2007). "Skype client using SQLite?" (<https://web.archive.org/web/20071117061133/https://www.mail-archive.com/sqlite-users%40sqlite.org/msg27326.html>). *sqlite-users* (Mailing list). Archived from the original (<https://www.mail-archive.com/sqlite-users%40sqlite.org/msg27326.html>) on 2007-11-17. Retrieved June 14, 2010.

66. "WhatsApp in Plain Sight: Where and How You Can Collect Forensic Artifacts" (<https://www.group-ib.com/blog/whatsapp-forensic-artifacts/>). *www.group-ib.com*. 7 November 2019. Retrieved 29 September 2025.
67. "Addendum: Project Years of Expenses With Quicken for Mac" (<https://frugalvagabond.com/addendum-project-years-of-expenses-with-quicken-for-mac/>). *The Frugal Vagabond*.
68. "sqlite3 — DB-API 2.0 interface for SQLite databases" (<https://docs.python.org/library/sqlite3>). *The Python Standard Library Documentation*.
69. "Wine 11.9 Released" (<https://www.winehq.org/news/2026051501>). *WineHQ*. 15 May 2026. Retrieved 19 May 2026.

Sources

- Allen, Grant; Owens, Mike (November 5, 2010). *The Definitive Guide to SQLite* (<https://web.archive.org/web/20101230035043/http://apress.com/book/view/1430232250>) (2nd ed.). Apress. p. 368. ISBN 978-1-4302-3225-4. Archived from the original (<http://apress.com/book/view/1430232250>) on December 30, 2010. Retrieved December 23, 2010.
- Kreibich, Jay A. (August 17, 2010). *Using SQLite* (<http://oreilly.com/catalog/9780596521196>) (1st ed.). O'Reilly Media. p. 528. ISBN 978-0-596-52118-9. Archived (<https://web.archive.org/web/20101225102001/http://oreilly.com/catalog/9780596521196>) from the original on December 25, 2010. Retrieved December 23, 2010.
- Newman, Chris (November 9, 2004). *SQLite (Developer's Library)* (<http://www.informit.com/store/product.aspx?isbn=067232685X>) (1st ed.). Sams. p. 336. ISBN 0-672-32685-X. Archived (<https://web.archive.org/web/20120114075902/http://www.informit.com/store/product.aspx?isbn=067232685X>) from the original on January 14, 2012. Retrieved May 12, 2010.

Further reading

- Gaffney, Kevin P; Prammer, Martin; Brasfield, Larry; Hipp, D Richard; Kennedy, Dan; Patel, Jignesh M (2022-08-01). "SQLite: past, present, and future" (<https://dl.acm.org/doi/10.14778/3554821.3554842>). *Proceedings of the VLDB Endowment*. **15** (12): 3535–3547. doi:10.14778/3554821.3554842 (<https://doi.org/10.14778%2F3554821.3554842>).

External links

- Official website (<https://sqlite.org>) 
 - "The Untold Story of SQLite" (<https://corecursive.com/066-sqlite-with-richard-hipp/>). CoRecursive.
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=SQLite&oldid=1364362807>"